

Received 30 December 2025, accepted 24 January 2026, date of publication 28 January 2026, date of current version 3 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3658719

APPLIED RESEARCH

SHIFT: Self-Healing Intelligence in Feature Store and Data Store Transitions

SEONG-HYUN SEO AND DONG-JOON LIM^{ID}

Department of Industrial Engineering, Sungkyunkwan University, Suwon 16419, South Korea

Corresponding author: Dong-Joon Lim (tgno3@skku.edu)

This work was supported in part by the Ministry of SMEs and Startups of Korea (MSS), South Korea, under Grant RS-2025-25461622; and in part by the Hyundai Motor Chung Mong-Koo Foundation.

ABSTRACT This study presents an AI model management framework for maintaining the prediction performance of virtual metrology (VM) systems in steel manufacturing processes. The proposed method addresses concept drift (CD), a major threat to long-term model reliability, by employing a dual-update strategy that autonomously updates both the feature store and the data store. Unlike conventional model maintenance approaches based on static inputs and accumulated data, our framework continuously reassesses feature relevance and adapts retraining datasets to reflect evolving process conditions. In a real-world steel sintering process, the method achieved significant recovery in predictive accuracy—from 65% post-deployment to over 93% after updates. This study highlights the importance of proactive maintenance for AI models and demonstrates how an intelligent update strategy can extend the usable life and reliability of VM systems in critical production workflows. The findings contribute to the broader field of AI-enabled asset management by offering a scalable solution for condition monitoring, retraining scheduling, and long-term performance assurance.

INDEX TERMS Operational intelligence, virtual metrology, autonomous model update, concept drift, data store, feature store.

I. INTRODUCTION

In various industrial domains, the performance of deployed artificial intelligence (AI) models has a direct impact on operational decisions. When these models experience unexpected degradation, the consequences can include maintenance errors, production downtime, resource misallocation, and a loss of stakeholder trust. For instance, in high-throughput manufacturing environments, slight drifts in data patterns can lead to inaccurate yield predictions or delayed fault detection, undermining the reliability of automated maintenance decisions. As AI models become more deeply integrated into the operational fabric of engineering systems, their sustainability and long-term reliability have become pivotal concerns for asset-intensive industries.

A major factor influencing the long-term reliability of AI models in dynamic environments is concept drift (CD)—a

phenomenon where the statistical properties of input data and their relationship to target variables change over time [1]. CD can arise due to changes in system behavior, environmental conditions, or upstream data-generating processes. Without timely adaptation, models trained on historical data gradually lose their predictive accuracy, leading to suboptimal or even erroneous maintenance actions. Addressing CD is thus essential to maintaining the integrity and trustworthiness of AI-enabled maintenance frameworks, particularly when these models are tasked with representing and sustaining virtualized engineering assets.

Despite growing awareness of CD, many existing retraining strategies for predictive maintenance rely on fixed feature sets and static data selection mechanisms. These approaches often involve periodic retraining using either a sliding window of recent data or an ever-growing historical dataset, without reevaluating the relevance of input features or the underlying data quality. Such inflexibility fails to account for evolving system dynamics and may result in overly narrow

The associate editor coordinating the review of this manuscript and approving it for publication was Sangsoo Lim^{ID}.

or outdated models. In high-stakes engineering contexts, this rigidity limits the model's adaptability and degrades its utility as a robust virtual asset.

This study introduces an AI model management framework that jointly coordinates dynamic updates to both the feature store and the data store in response to CD. The proposed approach operationalizes two complementary capabilities: 1) *feature store reconfiguration*, which continuously reassesses and revises the input representation as feature relevance changes, and 2) *drift-aware data store construction*, which curates retraining data to reflect the current operating regime rather than relying on immediate, uniformly sampled retraining data. These components are integrated into an automated retraining workflow that reduces manual intervention and aligns model updates with maintenance decision cycles. The practical value of the framework is validated through a real-world steel manufacturing case study, where it was applied to a virtual metrology (VM) model for sinter quality prediction and demonstrated stable post-deployment performance across shifting process conditions, supporting reliable model operation in complex industrial environments.

The remainder of this paper is structured as follows. Section II reviews key concepts in AI model management, with an emphasis on workflow orchestration strategies for CD detection and adaptation. Section III discusses the rationale for dynamically updating the feature store and data store during the retraining process. Section IV describes the proposed model update framework in detail. Section V presents empirical results from a case study in steel manufacturing, demonstrating the effectiveness of the approach. Finally, Section VI concludes the paper and outlines directions for future research.

II. BACKGROUND

A. CONCEPT DRIFT

CD refers to the phenomenon where the statistical properties of the predictor variables and/or the target variable change over time [2]. This change can significantly impact the performance of AI models as the assumptions on which the model was trained may no longer hold. Common causes of CD include 1) changes in the environment or underlying processes generating the data, 2) changes in user preferences or behaviors, and 3) the evolution of external factors influencing the relationship between variables.

The sources of CD are theoretically categorized into three types [3], [4]:

- 1) *Virtual (or covariate) drift*: This refers to significant changes in the distribution of predictor variables, while the relationship between the predictor variables and the target variable remains unchanged.
- 2) *Real (or class) drift*: This refers to significant changes occurring in the relationship between predictor variables and the target variable while the distribution of the predictor variables does not change materially.

- 3) *Mixed drift*: This refers to significant changes in both the distribution of the predictor variables and their relationship with the target variable.

Research on CD can be divided into two procedural stages: CD detection and CD adaptation. The former focuses on identifying the occurrence of CD, thereby determining the proper timing for AI model updates, essentially addressing the question of “when” to update the model. The latter involves designing the optimal method for updating the model, addressing the question of “how” to implement these updates.

This detection–adaptation cycle mirrors traditional maintenance planning in engineering systems, where condition monitoring informs the scheduling of repairs or replacements. In this light, CD management can be seen as a digital parallel to preventive maintenance—ensuring the sustained performance and reliability of AI assets under changing operational conditions.

B. CD DETECTION

The most definitive method to detect CD may be to measure whether there are significant changes in newly incoming data. Conventional statistical process control based methods [5] have been extensively studied and implemented in the literature. Notably, the cumulative sum control chart [6] and various time series forecasting methods [7] have proven effective in identifying shifts in the mean of a data stream. More rigorous statistical approaches have gained traction due to their ability to detect varying data stream dynamics. The Kullback-Leibler divergence [8] and Kolmogorov-Smirnov test [9] are prominent distribution-based techniques that measure the divergence between probability distributions, providing a statistical basis for detecting changes. Window-based methods such as sliding windows focus on comparing data within different time frames. This approach allows for straightforward comparisons between current and previous data distributions. Furthermore, hypothesis test-based methods like the Page-Hinkley test [10] utilize sequential analysis and statistical hypothesis testing to identify shifts. These methods collectively enhance the ability to detect and respond to CD in dynamic data environments.

More recently, information-theoretic methods have been leveraged to measure changes in the dependence between variables and the uncertainty within data distributions. In general, mutual information and/or entropy are utilized to identify changes in data distribution. MI quantifies the degree of dependence between two variables, allowing for the detection of changes in their relationship. Entropy-based methods, in contrast, quantify the uncertainty or randomness of a distribution. k-dimensional quantization (KDQ) trees [11], an advanced method in this category, partition the data space into hyperrectangles and recursively split these regions based on data density and distribution characteristics. By monitoring the entropy within these partitions, KDQ trees can detect shifts in the underlying data distribution, signaling potential

CD. This approach is effective in capturing both global and local changes, making it particularly suited for complex, high-dimensional data streams.

Performance-based CD detection methods directly monitor the performance metrics of the AI models rather than relying on the statistical properties of the data stream. Methods within this category typically observe indicators such as error rates, accuracy, and classifier confidence to detect shifts in the data distribution that may degrade model performance. A representative example is the drift detection method (DDM) [12], which tracks the error rate of a classifier and signals a drift when significant deviations from the expected error rate occur. Similarly, the early drift detection method (EDDM) [13] extends DDM by focusing on the distance between classification errors, enhancing sensitivity to gradual drifts. Another noteworthy method is adaptive windowing (ADWIN) [14], which dynamically adjusts the size of the data window based on observed changes in error rates, providing a balance between responsiveness to new data and stability.

Recent research on CD detection has introduced novel approaches that focus on changes in the importance of individual features, moving beyond traditional methods that solely monitor overall model performance. These new techniques aim to detect CD earlier and more precisely by tracking shifts in the influence of each feature on model predictions. A prominent example is the feature-based drift detector (FBDD) [15], which employs LASSO regression to calculate the importance ranking of features in each data chunk and continuously monitors changes in the rankings of key variables. Similarly, feature-based analysis (FBA) [16] combines Kullback-Leibler divergence with game-theoretic Shapley values to quantitatively assess each feature's contribution to drift, enabling the identification of primary drift-inducing variables. Other methods such as multilayer concept drift detection method based on model explainability (HCDD-SHAP) [17] and SHAP-ADWIN [18] analyze temporal patterns in SHAP values to distinguish between genuine concept drift and transient data noise by observing fluctuations in feature attribution scores over time. Additionally, the CD detection and rationalization method (CDDRM) [19] leverages the non-combinatorial optimization via trace exponential and augmented Lagrangian for structure learning (NOTEARS) [20] algorithm to learn causal structures and detect changes in causal dependencies among variables, proposing a framework that categorizes drift into four distinct types based on their causes and characteristics.

C. CD ADAPTATION

CD primarily occurs when AI models operate in dynamic environments. This phenomenon arises because changes in data distribution over time can directly impact the performance of pre-trained AI models. Therefore, it is essential to implement adaptive model management that can account for these temporal changes. Strategies to address CD should

include methods for managing learning models, training data, and new data in dynamic environments. Furthermore, these strategies should be designed to balance the knowledge gained from prior training with the new insights derived from learning with fresh data [21]. The greater the emphasis on prior knowledge, the more the strategy prioritizes stability; conversely, greater emphasis on new knowledge increases adaptability.

Based on the interplay between prior and new knowledge, existing research distinguishes between two primary approaches for addressing CD: the blinded approach and the informed approach [22], [23]. The blinded approach entails retraining the model at predetermined intervals, such as fixed time periods, without leveraging any information regarding detected CD. In scenarios where data arrival is irregular, this method may instead trigger retraining once a predefined volume of new data has been accumulated, rather than adhering to fixed temporal intervals. Conversely, the informed approach adopts a more reactive stance, as it incorporates model retraining strategies explicitly designed to enhance adaptation to identified CD.

In the context of CD adaptation, a data management strategy primarily concerns how to structure the data for retraining the AI model. To effectively respond to CD, it is essential to make optimal use of the most recent data where changes have occurred [24]. There are two primary approaches to structuring retraining data. The first approach emphasizes the continuity of the data stream by synthesizing a dataset that combines a segment of prior knowledge with new knowledge. This method leverages not only the new knowledge but also the most recent prior knowledge, thus utilizing data that is directly relevant to the detected CD. The second approach involves composing the retraining dataset by combining recent data, where CD has been detected, with past data segments that exhibit similar characteristics. By using these similar past data segments as a reference, this method allows the ML model to adopt appropriate adaptation strategies drawn from historical data.

Another key aspect of CD adaptation is the implementation of a robust model management strategy. This strategy includes essential activities such as versioning, monitoring, and governance of models to ensure they remain effective, reliable, and scalable in production environments [25]. Effective model management also involves tracking model changes, monitoring real-time performance, and ensuring compliance with regulatory standards. This approach is vital in mitigating risks associated with model drift, which may arise as data and operating conditions evolve.

Recent CD adaptation methods are increasingly leveraging ensemble approaches that dynamically adjust model weights based on each predictive model's performance, thereby ensuring stable prediction accuracy despite shifts in data patterns. A representative example is the on-line weighted ensemble (OWE) [26], which employs a boosting strategy to update weights according to prediction errors on both recent and past data; this strategy discards underperforming models and

introduces new ones better suited to assimilate novel data, thereby enhancing adaptability. Dynamic and on-line ensemble regression (DOER) [27] adopts a similar principle within a sliding window framework, assigning weights to each model according to its mean squared error, so that higher-performing models receive greater emphasis. Likewise, the online accuracy updated ensemble (OAUE) [28] combines block-based learning and online learning to dynamically adjust classifier weights based on performance, replacing underperforming classifiers to accommodate various types of CD.

In addition, some methods integrate both data management and model management to enable models to effectively handle emerging data distributions. For instance, the adaptive sliding window based drift detection method (ADDM) [29] utilizes information entropy and the Hoeffding Bound to dynamically adjust the sliding window size, collecting data after CD occurs to determine the optimal retraining point. Similarly, CD type identification based on multi-sliding windows (CDT_MSW) [30] identifies the location and type of CD using multiple sliding windows and continues to accumulate data following CD detection to stably adapt to the new data distribution.

D. WORKFLOW ORCHESTRATION

Monitoring AI models in production is essential because the performance of a model can degrade over time due to various factors, such as data drift, model drift, or changes in the underlying distribution of input data [31]. Workflow orchestration tools enable continuous monitoring of model performance by integrating real-time analytics and performance metrics tracking. This continuous feedback loop allows the detection of CD or significant deviations in model behavior promptly. Without proper model monitoring, models that once performed well can start producing inaccurate or biased predictions, leading to potential business risks and a loss of trust in data-driven systems.

The role of workflow orchestration extends beyond monitoring; it also facilitates the seamless updating of AI models. When a model's performance deteriorates, it may require retraining with new data or even a complete overhaul of its architecture. Workflow orchestration tools can automate the retraining process by scheduling appropriate updates, managing data pipelines, and ensuring that the retrained model is rigorously tested before redeployment [32]. This ability to automatically trigger updates is crucial in maintaining the relevance and accuracy of AI models, especially in environments where data is constantly evolving.

Furthermore, workflow orchestration supports the versioning of models and data, which is vital for tracking changes and understanding the impact of updates. By keeping a record of each version of a model and the data it was trained on, it becomes possible to compare the performance of different models and revert to a previous version if necessary. This versioning capability also aids in compliance with

regulations that require transparency and accountability in AI systems. Workflow orchestration ensures that each update is well-documented, reproducible, and auditable, thereby fostering a culture of continuous improvement and quality assurance in smart systems.

Two key components that enable such version management are the feature store and the data store, which provide a systematic foundation for recording and managing changes in both the model and training data. The feature store serves as a structured repository for input features used in modeling. By organizing and storing feature combinations, importance rankings, and metadata, it helps maintain consistency in model training and inference workflows. The data store, on the other hand, retains both raw data and processed input variables, thereby providing the basis for defining the scope and composition of datasets used for model retraining. Together, these two storage systems not only support model reproducibility and transparency but also facilitate prompt and justifiable model updates in response to CD. When integrated into workflow orchestration, they contribute to sustained system reliability and continuous improvement in operational environments.

In summary, workflow orchestration ensures the long-term viability and effectiveness of AI models in production. By integrating robust monitoring and automated updating mechanisms, workflow orchestration tools help organizations maintain high-performance models that adapt to changes in data and business needs. This continuous cycle of monitoring, updating, and optimizing models is crucial for leveraging the full potential of AI while mitigating risks associated with model degradation and CD.

III. MOTIVATION

In this section, we highlight a critical gap in conventional approaches to reliability and maintenance in AI-enabled systems. While prior work has addressed model performance degradation, relatively little attention has been paid to systematic strategies for updating the feature store and data store—both of which are essential for sustaining model relevance over time.

A. FEATURE STORE UPDATES

Feature store updates are a critical aspect of updating AI models to adapt to CD, in which the statistical properties of the input data change over time. While the standard approach in many previous studies has been to address CD by simply retraining the model on new data, this method often operates under the rigid assumption that the initial set of features remains universally relevant throughout the model's lifecycle [12], [14], [33], [34]. However, this assumption is frequently violated in dynamic environments where the relationships between features and the target variable evolve. By failing to re-evaluate or purge feature sets during the update process, these studies continue to rely on outdated inputs, leading to a structural mismatch between the model and the current data stream that undermines long-term reliability.

In addition, focusing solely on retraining with a static feature set fails to account for shifts in importance or the emergence of new predictive signals. Although advanced methods may detect CD via feature-level changes, they typically lack a systematic orchestration mechanism to translate these detections into actual pipeline updates. This reveals a significant structural limitation: by decoupling drift detection from feature management, existing frameworks omit the necessary downstream adaptation phase, so fine-grained monitoring rarely results in tangible performance improvement. Consequently, models are constrained by their initial design, failing to capture the true underlying patterns as the data landscape shifts.

To bridge these gaps, this study proposes a new model management framework that integrates autonomous feature selection into the model update orchestration. Rather than relying on a fixed input structure, the proposed method utilizes a novel algorithm to dynamically reconfigure the feature store whenever drift is detected, prioritizing variables with the highest relevance to current operational states. By synchronizing feature store updates with real-time detection outcomes, our approach ensures that the retraining process not only incorporates fresh data but also adaptively expands or reshapes the model's representational capacity to reflect complex system dynamics.

B. DATA STORE UPDATES

Configuring retraining data is a critical step for adapting AI models to CD, where the data-generating process evolves over time. However, many prior studies emphasize when to retrain (e.g., immediately after CD detection) while under-specifying which data should be used for retraining [35], [36], [37]. As a result, “instant retraining” can proceed with minimally curated datasets, overlooking whether recent samples are truly representative, whether outdated segments should be removed, or whether newly available data streams should be incorporated.

A key limitation of this practice is the implicit assumption that the data pipeline can remain unchanged regardless of the drift regime. In reality, retraining on poorly aligned data can either lag behind the new concept (by over-retaining obsolete history) or overreact to short-lived disturbances (by amplifying noise and anomalies). Effective data configuration therefore requires drift-aware curation—such as selecting appropriate temporal windows, filtering transient outliers, and verifying that the retained data quality and distribution match the current operating conditions.

Accordingly, data store updates should be treated as an indispensable element of the model update workflow. Instead of retraining with whatever data happen to be available at the trigger time, the retraining dataset must be explicitly constructed to reflect the post-drift regime. This shift in focus—from retraining timing alone to retraining data relevance and quality—directly addresses the narrow scope of earlier approaches and enables more robust performance maintenance under CD.

To address these limitations, this study proposes a framework that strengthens data store management via an adaptive buffer period and differential weighting. Instead of triggering immediate, uniformly sampled retraining, the method first accumulates a drift-relevant post-change segment to form a training window that better reflects the current operating regime. It then applies a weighting scheme that emphasizes recent, informative trends while down-weighting legacy samples and transient fluctuations, thereby improving robustness against short-lived anomalies. Through this drift-aware orchestration of retraining data, the updated model can track emerging patterns more reliably and sustain post-deployment performance with more targeted and defensible updates.

Table 1 presents a comparative overview of the notable CD detection and/or adaptation methods in the literature. The column labeled ‘Multiple windows’ shows whether the method detects CD from more than one time window. The column titled ‘Multiple learners’ specifies whether the method uses more than one learner for CD detection. Representative methods based on both multiple windows and multiple learners—such as PL [38] and AL [39]—detect CD by comparing the error rates of two distinct learners. However, these approaches rely solely on differences in predictive performance, without considering changes in feature importance or shifts in the internal structure of the model. As a result, they overlook interpretability-related aspects of model change, such as explainability or structural variation.

‘Feature store update’ indicates whether the method evaluates and updates multiple feature sets in detecting CD. The column named ‘Buffer’ reflects whether the method updates the model after accumulating data following CD detection instead of retraining immediately upon detecting CD. While DDM [12] and EDDM [13] collect pre-drift data as a buffer and retrain immediately after CD detection, they do not incorporate the importance of newly arrived data or implement any structural updates. As a result, their ability to respond effectively to sudden drift remains limited.

Lastly, ‘Differential weighting’ denotes whether the method incorporates the significance of the recent period, where CD was detected, into the model update process. Online ensemble-based methods such as OWE [26] and OAUE [28] apply weighting strategies based on past predictive performance within an ensemble framework. However, since these weights are determined by performance under previous concepts, the models may fail to quickly adapt to newly emerging data distributions or conceptual changes. This overreliance on historical information can hinder responsiveness and potentially lead to a decline in predictive accuracy in the presence of CD.

To the best of our knowledge, existing workflow orchestration methods do not provide an integrated mechanism that simultaneously supports feature store updates, data store updates, and multi-window monitoring with multiple learners. In the following sections, we elaborate on the rationale behind this design and present the corresponding modeling and orchestration solutions in detail.

TABLE 1. Comparison of the proposed method with the existing CD detection and/or adaptation methods.

Method	Multiple windows	Multiple learners	Feature store updates	Data store updates	
				Buffer	Differential weighting
SHIFT (This study)	○	○	○	○	○
OWE [26]	○	○	×	×	○
OAUE [28]	○	○	×	×	○
DOER [27]	○	○	×	×	○
fsNB [40]	○	○	×	○	×
AE-LSTM [41]	○	○	×	×	○
MWDDM-H/M [42]	○	×	×	○	○
CDT_MSW [30]	○	×	×	○	×
ADDM [29]	○	×	×	○	×
PL [38]	○	○	×	×	×
AL [39]	○	○	×	×	×
STUDD [43]	○	○	×	×	×
FBDD [15]	○	×	×	○	×
OEOA [44]	×	○	×	×	○
FSDD [45]	○	×	×	×	×
STEPD [33]	○	×	×	×	×
FBA [16]	○	×	×	×	×
CDDRM [19]	○	×	×	×	×
DDM [12]	×	×	×	○	×
EDDM [13]	×	×	×	○	×
RDDM [46]	×	×	×	○	×
HCDD-SHAP [17]	×	×	×	○	×
QT [47]	×	×	×	○	×
MDDM [48]	×	×	×	×	○
MDDM-A/G/E [48]	×	×	×	×	○
FW-DDM [49]	×	×	×	×	○
BDDM [50]	×	×	×	×	○
FHDDM [51]	×	×	×	×	×
HDDM-A/W [52]	×	×	×	×	×
ADWIN [14]	×	×	×	×	×
SHAP-ADWIN [18]	×	×	×	×	×
ECDD [53]	×	×	×	×	×
EDIST 1&2 [34], [54]	×	×	×	×	×
SeqDrift 1&2 [35], [55]	×	×	×	×	×
SEED [36]	×	×	×	×	×
PERM [56]	×	×	×	×	×
DMWM [56]	×	×	×	×	×

IV. METHODOLOGY

This section provides a detailed explanation of the proposed method, SHIFT (Self-Healing Intelligence in Feature store and data store Transitions), which comprises two key processes: CD detection and adaptation as illustrated in Figure 1.

A. CD DETECTION

Existing CD detection methods are based on various approaches to address CD. Among these, methods utilizing multiple windows and multiple learners typically employ two learning models: a long-term memory model and a

short-term memory model [33], [38], [39], [40], [43]. The long-term memory model (L) generally learns stable concepts by leveraging a long period of historical data, while the short-term memory model (S) focuses on learning the most recent, temporary concepts using a small, recent data window. By monitoring the performance difference between the two models, CD can be detected, thereby prompting model updates or resets as necessary.

However, there are several limitations to the existing CD detection methods based on multiple windows. First, the combination of predictor variables is generally fixed, which

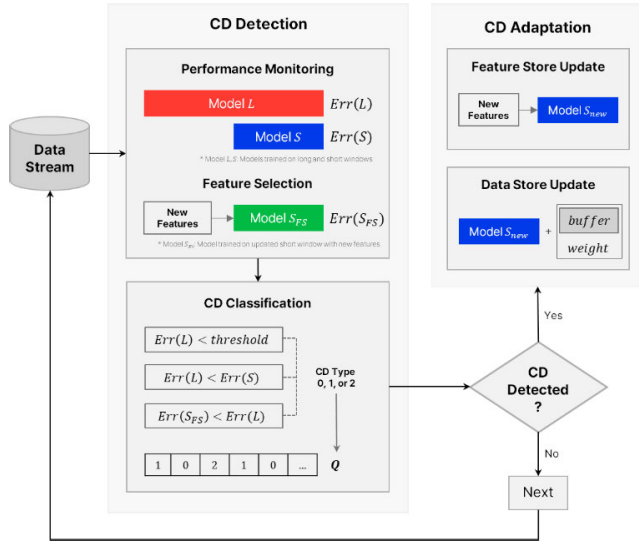


FIGURE 1. Overview of the proposed method: SHIFT.

TABLE 2. Descriptions for notations.

Notation	Description
$data (= \{(x_t, y_t)\}_{t=1}^T)$	Input dataset consisting of T pairs of features and target values
$batch$	Number of data points evaluated in each iteration (t)
l, s	Endpoints of the long-window (l) and short-window (s) training periods
L, S, S_{new}	Predictive models trained on long, short, and updated short windows
$E_L, E_S, E_{S_{new}}$	Lists of prediction errors collected from models L, S , and S_{new}
F, F_{new}	Feature sets for model training; F_{new} is a newly selected variable combination upon drift
τ	Minimum acceptable performance for L ; controls drift detection sensitivity
δ	Density threshold for determining drift types based on Q patterns
n	Minimum length of Q before evaluating for CD
q_{max}	Maximum size of Q to prevent over accumulation and maintain sensitivity of Q
$buffer$	Time interval to accumulate data post-drift before retraining begins; balances speed and robustness.
w	Assigns differential weights to $buffer$ to emphasize recent trends.

prevents the method from fully reflecting the impact of specific features on detecting CD. This can be problematic, as various features may play a crucial role in a dynamically changing environment, yet their influence is not adequately considered.

Second, process-change patterns are only coarsely modeled, making it difficult to flexibly respond to diverse change scenarios. For instance, in the event of a sudden environmental shift, existing frameworks of alternating learning

Algorithm 1 SHIFT Algorithm

```

Require:  $data (= \{(x_t, y_t)\}_{t=1}^T), l, s, batch, F, \tau, n, \delta, q_{max}, buffer, w$ 
1 Initialize:  $Q \leftarrow ()$ 
2 while  $l + batch \leq T$  do
3   Train  $L$  on the dataset  $\{(x_t, y_t)\}_{t=1}^l$  with  $F$ 
4   Train  $S$  on the dataset  $\{(x_t, y_t)\}_{t=s}^l$  with  $F$ 
5    $E_L \leftarrow [], E_S \leftarrow [], E_{S_{new}} \leftarrow []$ 
6   for  $t = l + 1$  to  $l + batch$  do
7      $\hat{y}_t^L \leftarrow L(x_t), \hat{y}_t^S \leftarrow S(x_t)$ 
8      $E_L \leftarrow E_L \cup Error(y_t, \hat{y}_t^L), E_S \leftarrow E_S \cup Error(y_t, \hat{y}_t^S)$ 
9   end for
10  if  $mean(E_L) < \tau$  then
11     $Q \leftarrow \text{append}(Q, 0)$ 
12  else if  $mean(E_L) < mean(E_S)$  then
13     $F_{new} \leftarrow \text{FEATURE SELECTION on } (x_t, y_t)_{t=s}^l$ 
14    Train  $S_{new}$  on the dataset  $\{(x_t, y_t)\}_{t=s}^l$  with  $F_{new}$ 
15    for  $t = l + 1$  to  $l + batch$  do
16       $\hat{y}_t^{S_{new}} \leftarrow S_{new}(x_t)$ 
17       $E_{S_{new}} \leftarrow E_{S_{new}} \cup Error(y_t, \hat{y}_t^{S_{new}})$ 
18    end for
19    if  $mean(E_L) \geq mean(E_{S_{new}})$  then
20       $Q \leftarrow \text{append}(Q, 1)$ 
21    else
22       $Q \leftarrow \text{append}(Q, 0)$ 
23    end if
24  else
25     $Q \leftarrow \text{append}(Q, 2)$ 
26  end if
27  if  $len(Q) \geq n$  and  $sum(Q)/len(Q) \geq \delta$  then
28    if  $count(Q = 1) \geq count(Q = 2)$  then
29      Train  $L$  &  $S$  on  $\{(x_t, y_t)\}_{t=s}^{l + batch + buffer}$ 
30      with  $F_{new}$  and  $w$ 
31       $Q \leftarrow ()$ 
32    else
33      Train  $L$  &  $S$  on  $\{(x_t, y_t)\}_{t=s}^{l + batch + buffer}$ 
34      with  $F$  and  $w$ 
35       $Q \leftarrow ()$ 
36    end if
37     $l \leftarrow l + batch + buffer$ 
38     $s \leftarrow s + batch + buffer$ 
39  else if  $len(Q) > q_{max}$  then
40     $Q \leftarrow ()$ 
41     $l \leftarrow l + batch, s \leftarrow s + batch$ 
42  else
43     $l \leftarrow l + batch, s \leftarrow s + batch$ 
44  end if
45 end while

```

windows may struggle to quickly adapt. Lastly, as the long-term memory window accumulates, there is a potential for a decline in model sensitivity. When historical data builds up in the long window, the model may become overly reliant on older data rather than recent data, thereby reducing its responsiveness to new changes.

To overcome the aforementioned limitations and enhance the performance of CD detection, this study expands the CD detection patterns and defines model update strategies tailored to the identified pattern. The extended framework is formalized in Algorithm 1. Specifically, CD is first detected by monitoring the difference in the model's

predictive performance (L vs S), and three main patterns are employed to respond:

- 1) *Stationary*: If L maintains stable performance, the existing retraining approach is applied, where L is updated with the accumulated data, and S is updated with the most recent window. Stable performance is defined as the model meeting a predefined performance threshold (τ), as described in lines 10–11. Here, τ is a domain-calibrated robustness criterion that reflects the acceptable level of predictive error for reliable operation. For example, τ can be set at a point where the prediction error—accounting for measurement uncertainty—would imply a potential process-quality failure (i.e., an error magnitude beyond which the system's decisions may become unreliable). This outcome is logged in Q with a value of 0.
- 2) *Real drift*: When the performance of L falls below the predefined threshold but remains superior to S , a feature store update is initiated conditionally: if the performance of S_{new} , enhanced through feature selection, surpasses that of L . In this case, recent pattern shifts are considered a potential signal of real drift and addressed by updating S with a new feature set. This outcome is recorded in Q with a value of 1. Conversely, if the updated model fails to outperform L , the drift is deemed transient, and Q is assigned a value of 0. This process is described in lines 12–23.
- 3) *Virtual drift*: When the performance of L drops below the threshold and S demonstrates superior performance, it becomes justifiable to update the model using only recent data. This scenario can be interpreted as a virtual drift, wherein predictor variables with meaningful signals are mostly concentrated in recent data rather than distributed across the entire training dataset, while the underlying relationship between predictor and target variables remains stable. This outcome is documented in Q with a value of 2, as outlined in line 25.

It is important to emphasize that the final CD detection is determined through a queue management process. Specifically, the patterns described above serve as batch-level indicators of CD within each window (*batch*), and the ultimate detection point is identified only after accumulating and analyzing these results over a specific period. Until this determination is made, L continues to be updated with accumulated data, while S is updated with data from the most recent window.

Queue management offers two key advantages in controlling the sensitivity of CD detection:

- 1) *Robust to transient CD events*: It prevents the model from overreacting to temporary pattern changes, which could otherwise compromise its stability. Excessive sensitivity often necessitates more frequent retraining cycles, which increases computational overhead and resource consumption without corresponding improve-

ments in model performance. By adjusting the queue (Q), the final detection decision is made only after sufficient evidence of CD has been accumulated, ensuring a more reliable detection process.

- 2) *Resilience to transient CD patterns*: Not only can the occurrence of CD be temporary, but its patterns may also be short-lived. To determine the most appropriate retraining approach, it is essential to identify the dominant CD pattern over the queue period, allowing for more robust handling of fluctuating drift signatures.

In this study, we introduce a novel method that conditionally records 0, 1, or 2 in a dedicated vector Q and, upon reaching a specified threshold, determines whether the current data flow corresponds to stationary, real drift, or virtual drift. This approach addresses a key limitation of related methods such as alternating learners (AL) [39] which do not distinguish the scenario $\tau < \text{mean}(E_L) < \text{mean}(E_S)$ with $\tau > \text{mean}(E_L)$, effectively treating potential real drift and the stationary condition as identical. By explicitly separating these cases, the proposed framework enables tailored responses: maintaining computational efficiency through selective retraining under stationary conditions; optimizing feature combinations via feature selection in the presence of real drift; and performing focused updates using recent data in cases of virtual drift. These adaptive strategies, grounded in the specific characteristics of each drift type, offer a more efficient and targeted alternative to the uniform retraining procedures commonly employed in traditional methods such as AL and paired learners (PL) [38]. As a result, the proposed framework reduces unnecessary retraining costs while enhancing responsiveness to diverse forms of CD.

Unlike conventional methods that update the queue with every incoming data point, the proposed method performs batch-wise updates, enabling more careful monitoring and classification of three distinct patterns in the data stream. By accumulating sufficient observations within each batch, the framework reduces the risk of mistaking transient fluctuations for true drifts and improves the reliability of pattern-based drift classification. This, in turn, helps distinguish temporary noise from meaningful shifts, avoiding unnecessary retraining while enabling timely adaptation to genuine environmental changes.

B. CD ADAPTATION

Sufficient evidence of CD, assessed based on the length and density of Q , informs the decision for CD detection and the initiation of the CD adaptation phase for model retraining. Specifically, as detailed in lines 27–41, the accumulated patterns in Q guide the choice to either reconstruct the combination of predictor variables and deploy a new model trained on the latest data (line 29) or retrain the existing model with a focus on recent data changes (line 32).

To adjust the sensitivity of CD adaptation, we propose two mechanisms: the buffer period (*buffer*) and differential weighting (w). The buffer period represents the interval

during which additional data is collected before updating the model following the detection of CD. A shorter buffer period facilitates a rapid adaptation strategy, enabling prompt responses to detected CD. Conversely, a longer buffer period supports a more robust adaptation strategy, reducing sensitivity to transient CD. The buffer period can be optimized to align with the intrinsic characteristics of the dataset or dynamically adjusted based on the severity of the detected CD.

Although DDM [12] and its derivative methods, such as RDDM [46] and HDDM [52], distinguish between a warning level and a drift level, and use the data in between for retraining, they do not systematically define retraining intervals during CD adaptation or explicitly accommodate post-detection distributions. Such approaches, which rely on pre-drift data for retraining, inherently face limitations in capturing new patterns that emerge in the post-drift data stream. This limitation becomes particularly pronounced in environments experiencing sudden drift, where retraining is performed solely on data preceding the shift. Without an immediate response to these new patterns, the model struggles to effectively learn from the altered data distribution, ultimately undermining both the responsiveness and reliability of the adaptation process. In contrast, the buffer period proposed in this study is designed to robustly adapt to emerging pattern changes by appropriately managing incoming data after CD detection and establishing retraining windows. By adjusting this buffer period to align with the data distribution, one can control both the stability and responsiveness of the retraining strategy, distinguishing this approach from existing methods.

The second mechanism, differential weighting, involves assigning varying weights to the data within the retraining window during model updates. This method explicitly differentiates the influence of data collected during the buffer period. By applying various weighting functions, such as uniform, stepwise, linear, or logarithmic, the model's sensitivity to recent data distributions during updates can be modulated. Similar to the buffer period, the appropriate weighting function can be tailored to the nature of the dataset and the specific type of CD. Additionally, this approach could be extended to dynamically adjust the weighting based on evolving CD patterns, further enhancing its adaptability.

The method of applying differential weights to retraining data distinguishes itself in several ways. For example, ensemble-based retraining methods, such as OWE [26], DOER [27], and OAUE [28], employ a boosting strategy that assigns greater weights to models exhibiting superior predictive performance during the training interval. However, these methods primarily focus on adjusting weights among ensemble members, without directly identifying or emphasizing important instances within the training data itself. Such indirect approaches limit the model's ability to finely adapt to newly emerging patterns. In contrast, the differential weighting mechanism proposed in this study directly adjusts the influence of each data point during retraining, enabling a more precise and responsive adaptation to changes in the data

distribution. Although traditional time-series methods like WMA [58] and EM [59] also utilize differential weighting, they implement it as a blind update; they continue forecasting unconditionally without linking it to the CD detection process. In contrast, the differential weighting introduced in this study is designed to act as a mechanism to effectively respond to observed pattern changes when a CD occurs and to fine-tune retraining sensitivity for more informed updates.

Finally, as outlined in lines 36–37, setting a maximum allowable length for Q prevents the decay of retraining sensitivity. Specifically, by resetting Q when it exceeds a certain length (q_{max}), the process mitigates the adverse effect of accumulated stationary states suppressing the detection and adaptation of meaningful CD. From a data store management perspective, this process plays a crucial role in structuring the long-term memory window by preventing the permanent accumulation of historical data. This ensures that the model maintains an appropriate balance between sensitivity to new patterns and robustness against transient changes.

V. CASE STUDY

In this section, we present an empirical case study showing how the proposed model management method was applied to a VM system deployed in the steel manufacturing process of a globally leading anonymized company in Korea. The case illustrates how the method sustains post-deployment model performance and supports operational dependability over time.

A. BACKGROUND

A core process in steel manufacturing is blast furnace refining, which is generally divided into two major stages: sintering and coking. In the sintering process (Figure 2), iron ore, coke powder, and other materials are agglomerated into uniform particles to produce sinter, the primary feedstock for molten iron. High-temperature reactions with oxygen convert iron ore into iron while separating impurities into slag. Because iron ore quality and form vary by region, it cannot be charged directly into the blast furnace; it must first be standardized in both quality and particle shape. The resulting sinter must maintain consistent size and composition to support stable furnace operation and high-quality molten iron, and its composition ratio can be adjusted to produce steel for diverse applications. Accordingly, plants derive quality control indicators to monitor and manage sinter quality, as maintaining uniformity in ore quality and sinter characteristics is critical to steel production.

Many AI/ML-based studies conducted in the steel manufacturing domain have primarily focused on static data analysis and modeling. For example, there is a study [60] that improved predictive performance by removing non-predictive factors in advance using the Boruta algorithm and building a support vector machine (SVM)-based VM model on real-time steel industry data. While this approach partially addressed dynamic characteristics, it remained limited to static data analysis and was unable to respond

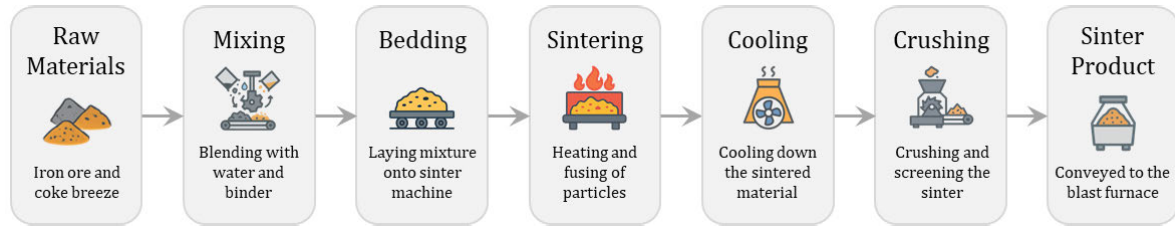


FIGURE 2. Overview of the sintering process in steel production.

to concept drift during operation. Similarly, another empirical study [61] employing Random Forest aimed to enhance energy prediction accuracy through the creation of derived variables and correlation-based feature selection; however, it also operated under a pre-defined static model structure. The dynamic modeling with memory (DMWM) framework [57] attempted to mitigate performance degradation by detecting drift based on MAPE and retraining the entire model upon detection. Although this approach achieved improved accuracy, it incurred increased computational cost due to full retraining and reduced adaptability due to discarding previously acquired knowledge during adaptation. Collectively, these studies highlight the pressing need for incremental and efficient continual learning frameworks capable of functioning effectively in dynamic environments such as steel manufacturing processes, where data characteristics evolve continuously over time.

In the company examined in this case study, a VM system is used to optimize operating conditions and manage sinter quality. In continuous steel manufacturing, direct real-time measurement of material properties is often impractical due to safety constraints and the high cost of physical inspection, making VM an essential operational alternative. The VM's core predictive model uses approximately 30 process variables to estimate sinter yield in real time and is therefore tightly embedded in the production workflow. However, when model performance degrades or anomalies occur, root-cause diagnosis is difficult because the process cannot be readily stopped, and replacing the model is rarely feasible. These constraints motivate an adaptive model management strategy that preserves reliability while supporting uninterrupted operation.

This case study is representative because it reflects a common, high-impact CD scenario in real-world VM deployment: continuous operation, limited real-time quality measurements, and a model that must remain in service as operating regimes evolve. In such settings, post-deployment degradation typically stems from accumulated shifts in data patterns and feature relevance rather than a single identifiable cause. The key challenge here was the absence of a structured retraining strategy and workflow orchestration to convert drift signals into actionable lifecycle steps (e.g., when to retrain, which data to prioritize, and whether to reconfigure features). As a result, the initially effective static model became unreliable on the shop floor, highlighting the operational necessity

of integrated CD detection and adaptation. By demonstrating how SHIFT coordinates feature/data store transitions to restore and sustain reliability, this case offers a practical orchestration template for mitigating performance decay in deployed industrial AI systems.

B. DATA

This case study utilizes time-series data collected every second over approximately 24 months (June 1, 2021–June 11, 2023) from a steel production process. In particular, the sintering process—where raw materials such as iron ore and coke powder react at high temperatures to produce sinter—is a critical phase because sinter serves as a primary input for blast furnaces. Given the strong influence of sinter quality on downstream steelmaking, diverse operational conditions and quality indicators were recorded for monitoring. The initial dataset comprised approximately 1.06 million instances and 127 features; after synchronizing operational conditions with target attributes, 874,342 valid entries were retained. For real-time monitoring and computational efficiency, the second-by-second data were aggregated into 10-minute intervals, yielding a final dataset of 50,832 instances with 127 features. This interval was determined with domain experts as a practical compromise: shorter intervals may capture patterns more precisely but increase computational cost, whereas longer intervals can suppress noise but risk obscuring meaningful signals. Accordingly, a 10-minute interval was selected to balance fidelity and operational feasibility.

The existing VM model was developed using Lasso regression. This choice was made in response to field requirements for a simple and interpretable model, prioritizing operational feasibility within the complex environment of steel production processes. While complex models tend to lack generalizability and are challenging to implement in operational settings, the Lasso model provided a suitable balance between predictive performance and interpretability. After model training and validation by domain experts, a total of 30 key variables out of the original 127 were utilized to predict the target quality attribute, the sinter production rate. The original model was developed using data spanning approximately one and a half years (from June 1, 2021, to December 31, 2022) and was deployed in the field in January 2023. Because no post-deployment model management policy was in place, the model was not retrained after deployment.

TABLE 3. Comparison of model performances after deployment.

	Metric	Predictive performance statistics				
		Overall (Jan.1. ~ Jun.11.)	Abnormal period 1 (Jan.31. ~ Feb.4.)	Abnormal period 2 (Feb.21. ~ Feb.27.)	Abnormal period 3 (Apr.18. ~ Apr.22.)	Abnormal period 4 (May.7. ~ May.10.)
Without model update (Original)	MAPE/CTQ	9.87/15.11	12.02/12.85	11.44/2.07	9.31/12.24	10.87/7.71
	SD of MAPE	0.85	1.01	0.50	0.60	0.91
	CI of MAPE	(1.86, 1.90)	(2.22, 2.42)	(2.12, 2.22)	(1.64, 1.75)	(2.00, 2.15)
With model update (This study)	MAPE/CTQ	2.14/93.12	2.41/90.42	1.83/94.08	1.94/94.92	2.06/91.19
	SD of MAPE	0.44	0.36	0.29	0.32	0.37
	CI of MAPE	(0.39, 0.41)	(0.41, 0.48)	(0.30, 0.35)	(0.32, 0.38)	(0.36, 0.42)

The model's performance was evaluated using mean absolute percentage error (MAPE) and critical to quality (CTQ) metrics. MAPE expresses the difference between predicted and actual values as a percentage. CTQ, on the other hand, measures the extent to which predictions fall within an acceptable range,¹ offering an intuitive reflection of field variations and quality control standards. For MAPE, lower values indicate better predictive performance as it represents error. Conversely, higher CTQ values signify better performance, as they indicate a greater proportion of predictions within the acceptable range. By employing both metrics, the evaluation provided both general predictive performance insights and a quality assessment easily understood and usable by domain experts.

C. ANALYSIS

1) PERFORMANCE DEGRADATION

The VM model, which initially promised a CTQ of 65% at the time of development, showed a marked decline in performance over time, reaching a point where it became challenging to rely on in real-world applications. As summarized in Table 3, the average CTQ declined to 15.11%, with four segments exhibiting high prediction variability, with CTQ values of 12.85%, 2.07%, 12.24%, and 7.71%, respectively (discussed later). In parallel, MAPE, as summarized in Figure 3, which was 4.56% during development, deteriorated to 9.87% after deployment and rose further to 12.02%, 11.44%, 9.31%, and 10.87% during high-volatility periods, critically undermining the model's reliability.

Figure 4 illustrates how the model's predicted values (blue line) systematically fall below the actual target values (black line), resulting in consistently negative errors. This bias skews the interpretation of predictive outcomes and erodes confidence in the model's decision-support capabilities. The emergence of such bias suggests the presence of structural deficiencies within the modeling process or overlooked factors in the incorporation of specific variables,

¹Even a minor difference in MAPE may sometimes constitute an unacceptable error in the field, as it can be classified as a CTQ criterion. In this case study, the allowable range was defined based on field requirements, corresponding to one standard deviation from the training data.

which have been revealed through the introduction of incoming data after deployment. Specifically, the model may fail to adequately capture the interactions between key predictor variables or might have omitted essential variables. As a result, the feature set that once performed well on training data fails to accommodate new data patterns, highlighting a common limitation of static models.

The post-deployment performance degradation underscores the critical need for a robust model management strategy that supports 1) recombination of predictor variables and 2) continuous monitoring of predictive metrics. An effective approach proactively tracks the model's accuracy and implements comprehensive model reconstruction, including both feature set revisions and parameter adjustments whenever anomalous patterns emerge. The following sections outline strategies to address these challenges by systematically updating the feature store and data store, ensuring a more resilient and adaptive VM system for sintering processes in steel production.

2) MODEL UPDATE STRATEGY

To address CD identified as real drift, we adopted a feature selection strategy that updates the feature store by combining minimum description length principle (MDLP) [62] and minimum redundancy maximum relevance (mRMR) [63]. MDLP discretizes continuous data, thereby reducing dimensionality and enhancing interpretability, whereas mRMR minimizes redundancy among variables and maximizes their correlation with the target variable. By applying mRMR to the data discretized through MDLP, we further reduce redundancy across variables while maximizing their relevance to the target variable.² Utilizing these two algorithms in tandem effectively narrows the feature space, prioritizing features

²Other well-known feature discretization and selection algorithms, including ChiMerge [64] and interaction weight-based feature selection (IWFS) [65] were also considered. However, in this case study, the combination of MDLP and mRMR proved to be the most appropriate for the given data. Furthermore, following consultations with domain experts, the feature selection process was specifically designed to include 11 essential features, preventing the model from relying on counterintuitive combinations of features solely to achieve predictive performance.

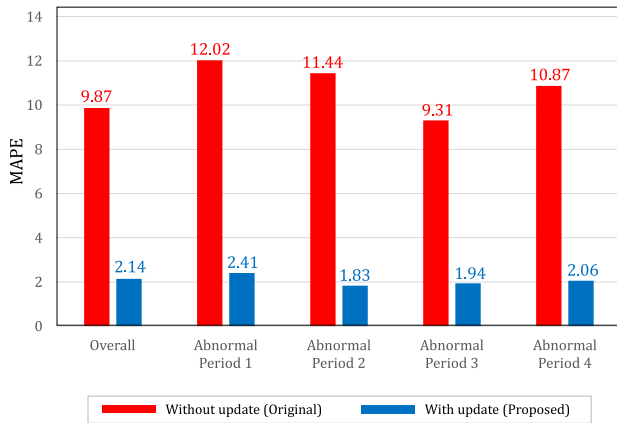


FIGURE 3. Comparison of model performance (MAPE).

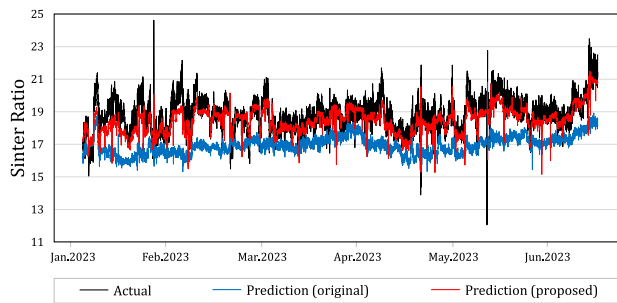


FIGURE 4. Comparison of predictive performance (predictions).

with the strongest associations to the target. Moreover, this combined method offers practical advantages, including low computational cost and fast execution speed, since it does not require repeated model training. Consequently, this approach not only reduces data complexity and improves interpretability but also proves highly effective in online learning environments where rapid response is essential.

The hyperparameters for the proposed method were determined by exploring various candidate values that aligned with the VM model's operating environment, ultimately selecting those yielding the best performance. The performance degradation threshold (τ) was set to correspond to MAPE values of 2, 4, and 6—levels considered practically meaningful in industrial forecasting contexts. The minimum length parameter (n) was tested with values of 2 and 4, selected to balance detection sensitivity and adaptation stability. The density threshold (δ) was set to 0.3, 0.5, and 0.7 to strike a balance between statistical reliability and responsiveness. Consequently, the window size of S was set to four hours, the performance verification interval (*batch*) to two hours, and τ for L was fixed at an MAPE value of 2. Additionally, n for the performance verification vector (Q) was set to 2, δ was set to 0.3, and Q was designed to reset once its length exceeded 7 (q_{max}).

Similarly, the hyperparameters for CD adaptation procedures, which regulate the sensitivity of the retraining strategy,

were determined through exploratory analysis. The buffer period was examined across a range from 6 to 18 hours to identify a value that balances the need for sufficient retraining data collection with the requirement for timely adaptation. We found that collecting six hours of post-detection data (*buffer*) was optimal for constructing the retraining dataset. To emphasize the relevance of recent data within the buffer period, various weighting strategies—such as linear, logarithmic, and double-weight schemes—were implemented and evaluated experimentally. During the six-hour buffer period, the data were assigned double weighting (w) in error calculations, thereby increasing the model's responsiveness to the most recent data.

Finally, the model update strategy was established using the parameters described above, and retraining was designed to occur autonomously based on the conditions specified by the proposed method, rather than on a fixed schedule. To assess its effectiveness, we monitored predictive performance from January through June 2023, assuming that the autonomous update strategy had been active since January 2023, the actual deployment date of the prediction model.

3) RESULTS

By applying the proposed method, it was demonstrated that the prediction model's average performance could be maintained at MAPE = 2.14% and CTQ = 93.12% after deployment. This outcome is a substantial improvement compared to the average performance observed without post-deployment retraining (MAPE = 9.87% and CTQ = 15.11%). In total, 621 data store data store updates were performed with dynamically adjusted retraining intervals, 395 of which involved feature store updates that reconfigured the predictor variables.

As a result, not only was the importance of the existing predictor variables updated through retraining, but nine new variables,³ which had not been considered during the initial model development, were frequently incorporated into the model during the retraining process. Notably, this update strategy enabled the model to exceed its original performance at the time of development (MAPE = 4.65% and CTQ = 65%), indicating that the model can adaptively evolve in response to changing data patterns.

The model update strategy exhibited an even more pronounced effect during abnormal periods. As summarized in Table 3, the performance that had sharply deteriorated after deployment was restored to a stable state, and in all intervals except abnormal period 1 (discussed in the following section), it even surpassed the overall average performance across the entire period in terms of both MAPE and CTQ. These findings demonstrate that the proposed model update strategy can effectively compensate for the performance degradation of the existing prediction model even when data pattern

³These newly introduced predictor variables include quicklime-to-concentrate ratio, alternating CaO content ratio, single-ore ratio, carbon content ratio, blower temperature, lime-to-concentrate ratio, charging ratio, basicity, charging density.

TABLE 4. Sensitivity analysis results of hyperparameters.

Detection hyperparameters			Adaptation hyperparameters		Predictive performance
τ	n	δ	buffer	w	MAPE/CTQ
2	0.3	2	6	Double	2.14/93.12
4	0.3	2	6	Double	2.29/92.52
6	0.3	2	6	Double	2.43/92.01
2	0.5	2	6	Linear	2.19/93.05
2	0.5	4	6	Linear	2.41/92.76
4	0.5	2	12	Linear	2.33/92.39
4	0.5	4	12	Linear	2.34/92.39
4	0.3	4	18	Double	2.26/92.64
4	0.5	4	18	Double	2.31/92.42
4	0.7	4	18	Double	2.33/92.30
6	0.3	2	18	Log	2.40/92.32
6	0.5	2	18	Log	2.41/91.60
6	0.7	2	18	Log	2.49/90.99

changes are especially severe. Overall, the proposed method ensures retraining is conducted in response to newly incoming data changes, thereby enhancing both the performance and stability of the prediction model. Its impact was particularly evident in periods where the original model struggled to produce accurate predictions.

Table 4 presents the combined effects of the hyperparameters τ , δ , n , *buffer*, and *weighting strategy* on predictive performance. The parameter τ controls the sensitivity of drift detection; lower values of τ lead to faster detection by reacting to even minor performance fluctuations, but at the cost of a higher false alarm rate. Conversely, higher values of τ enable more stable decision-making, though they may result in delayed detection. The parameter n defines the minimum number of consecutive detections required to confirm a drift, and its influence interacts with τ to affect overall sensitivity. Experimental results show that when τ is small, changes in n significantly impact predictive performance, whereas when τ is large, variations in n have minimal effect. This indicates that the influence of n is contingent upon the degree of detection sensitivity configured by τ . Additionally, δ , the threshold for the difference in error between two models, was found to reduce detection sensitivity as its value increases. Based on these findings, a low τ with a moderate n is recommended in highly dynamic environments requiring rapid drift response, whereas a higher τ with a conservative δ is preferable in stability-oriented settings to reduce false detections.

Performance gains were also statistically validated. Table 3 also contains the comparative analysis of prediction errors between the original model and the proposed update model in terms of standard deviation (SD) and 95% confidence intervals (CI). For the overall period, the confidence interval for the original model's prediction errors was estimated at (1.86, 1.90), indicating that the mean error is expected to

fall within this range with 95% probability. This suggests a relatively unstable prediction performance. In contrast, the proposed model achieved a much narrower and lower confidence interval of (0.39, 0.41), reflecting both lower error and reduced variance. This pattern was consistently observed across abnormal periods as well.

To statistically validate the significance of this difference, a paired *t*-test was conducted for the error distributions of the two models across all periods. In every case, the *p*-value was found to be below 0.00001, confirming that the performance difference is statistically significant. For the overall period, the *t*-statistic reached 181.50, providing strong evidence that the performance gap is not due to random sampling variability but rather a genuine effect. This result substantiates that the proposed model update strategy leads to a statistically meaningful enhancement in predictive performance.

In addition to the statistical tests, an analysis of the error distribution provides further insight into the behavioral differences between the models. Figure 5 compares the error distribution of the original model with that of the model operated using the proposed update strategy. As clearly illustrated, while the original model exhibits a negatively biased error distribution (underestimation), the retrained model, which has evolved through continuous updates, demonstrates a balanced distribution centered around zero. The absence of systematic bias in the error distribution is a crucial factor in assessing the reliability of the predictions in conjunction with their accuracy. These results reconfirm that the proposed model update strategy not only enhances overall predictive performance but also enables the model to capture critical data pattern changes that the original model failed to recognize, thereby ensuring more robust predictions.

The superior performance of the proposed model update strategy is further validated through comparisons with existing model management algorithms. As summarized in Table 5, all baseline methods, including DDM and its variants MDDM and FHDDM, which are based on error rate changes, ADWIN, which employs a sliding window approach, and STEPDP, which relies on statistical hypothesis testing, exhibited significantly lower accuracy than the proposed method across the entire evaluation period. Specifically, the methodologies based on multi-learner approaches, such as PL and AL, achieved relatively superior performance during some abnormal periods. However, in terms of overall performance across the entire evaluation period, the proposed method outperformed these methods.

To statistically validate performance differences between the proposed method and existing update strategies, we conducted the Wilcoxon signed-rank test. This non-parametric test evaluates not only average error levels but also the direction and consistency of per-time-step error differences. During the evaluation period, all methods were ranked at each time step based on prediction error, and average ranks were used for overall comparison. Each baseline was then compared pairwise against the proposed method. The significance level was set to 0.05, and Bonferroni correction was applied

TABLE 5. Comparative analysis of predictive performance across different model update strategies.

Method	Predictive performance (MAPE/CTQ)				
	Overall (Jan.1. ~ Jun.11.)	Abnormal period 1 (Jan.31. ~ Feb.4.)	Abnormal period 2 (Feb.21. ~ Feb.27.)	Abnormal period 3 (Apr.18. ~ Apr.22.)	Abnormal period 4 (May.7. ~ May.10.)
SHIFT	2.14/93.12	2.41/90.42	1.72/94.08	1.94/94.92	2.06/91.19
AL	2.42/91.41	3.03/85.51	1.79/94.97	1.69/97.46	2.66/86.61
DDM	2.43/91.21	3.11/84.11	2.01/92.01	1.72/97.46	2.26/92.66
FTDD	2.46/90.29	2.70/86.45	1.90/92.31	1.76/96.07	2.13/90.46
MDDM	2.48/90.25	3.27/84.35	1.85/93.20	2.10/91.69	2.36/90.09
WSTD	2.51/89.55	2.72/85.98	2.08/87.28	2.09/91.69	2.23/90.28
FHDDM	2.55/89.82	3.27/84.35	2.01/89.65	2.10/91.69	2.36/90.09
FHDDMS	2.57/89.65	3.27/84.35	1.97/91.12	2.10/91.69	2.43/88.07
ADWIN	2.58/89.24	3.16/84.58	1.80/92.31	1.87/94.46	2.46/88.07
STEPD	3.32/91.32	2.35/89.02	1.88/92.60	1.78/97.23	2.46/89.54
PL	3.41/91.99	2.72/86.92	1.56/95.86	2.28/97.23	2.17/89.54

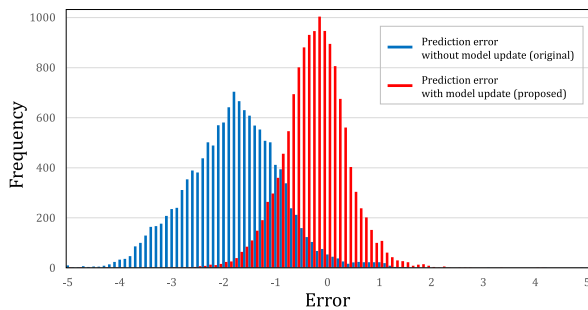


FIGURE 5. Prediction error ($\hat{y} - y$) comparison.

for multiple comparisons, yielding an adjusted threshold of approximately 0.0045. The test results are summarized in Table 6.

The results show that the proposed method significantly outperformed all existing update strategies except PL (consistent with Figure 6). It achieved the lowest average rank, indicating the best overall predictive performance. Although the rank-based difference with PL was not statistically significant, PL exhibited notably worse absolute accuracy in terms of MAPE (Table 5), suggesting larger errors in failure cases that may undermine reliability in practice. All other comparisons were statistically significant, supporting the proposed method's robustness and indicating that the improvements are consistent rather than due to random variation.

Despite the observed improvements in performance, the proposed method—comprising multiple computational components such as drift detection, reconfiguration of predictor variables, and retraining—requires validation for stable operation in real-world manufacturing environments.

To this end, we evaluated its computational complexity with a focus on time and memory efficiency. The results showed that the average processing time per data point was approximately 1.04 seconds. In cases where CD was detected and both the feature store and data store were updated for adaptation, the maximum processing time reached up to 100 seconds. Considering that new data is evaluated every 10 minutes in this application setting, these results indicate that the proposed method is sufficiently applicable for implementing workflow orchestration under online learning conditions. Furthermore, the peak memory usage throughout the entire experiment was approximately 1.27 GiB, demonstrating that the proposed approach can be feasibly deployed on computers commonly used by field operators in daily practice.

4) KEY OPERATING REGIMES

In this section, we examine the effectiveness of the proposed model update strategy in four abnormal periods that experienced severe performance degradation after deployment.

Abnormal period 1, located in the upper-left corner of Figure 7, can be classified as a sudden drift type, where multiple patterns occur irregularly. In this segment, the proposed model update strategy frequently performed retraining to adapt to changes in data patterns. Notably, during the sharp drop observed around February 2nd and 3rd, the model appeared to adapt effectively to the abrupt changes through proactive feature store updates. In the absence of retraining, significant performance degradation occurred (see Table 3), underscoring the critical role of retraining strategies in coping with highly uncertain data shifts. In terms of performance, the proposed method achieved MAPE = 2.41 and CTQ = 90.42, outperforming the other baseline methods.

TABLE 6. Results of Wilcoxon signed-rank test across different model update strategies.

Method	Mean Rank	<i>p</i> -value	Significance
SHIFT	5.41	-	-
PL	5.63	2.02E-2	FALSE
AL	5.94	2.22E-104	TRUE
STPD	5.97	4.55E-48	TRUE
FTDD	6.04	2.99E-79	TRUE
DDM	6.09	7.34E-63	TRUE
WSTD	6.18	1.11E-103	TRUE
ADWIN	6.25	1.16E-165	TRUE
MDDM	6.30	2.87E-103	TRUE
FHDDM	6.46	8.89E-144	TRUE
FHDDMS	6.46	1.21E-143	TRUE

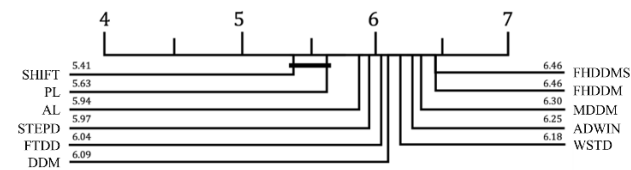


FIGURE 6. Critical difference diagram.

While the retraining strategy yielded comparatively better performance, sudden drift of this nature remains difficult to manage, even with adaptive mechanisms. Methods such as the DDM family and ADWIN exhibited relatively lower performance in this segment, likely due to their dependence on statistical monitoring of error rates or distributional shifts, which may have delayed their response. In the case of the DDM family, limitations exist not only in the drift detection process but also in the adaptation phase following detection. Specifically, DDM retrains the model using historical data prior to the detected drift point, which structurally prevents it from immediately incorporating newly emerged post-drift concepts. Failing to reflect post-drift data properly can lead to substantial drops in prediction accuracy or even incorrect decisions. In fact, DDM recorded MAPE = 3.11 and CTQ = 84.11, which are significantly lower than those achieved by the proposed method. In contrast, the proposed method exhibited greater adaptability by alternating between data store and feature store updates, enabling more effective accommodation of sudden changes by capturing diverse patterns in recent data.

Abnormal period 2, situated in the upper-right of Figure 7, represents a typical case of recurring drift, where temporary fluctuations in data patterns occur before returning

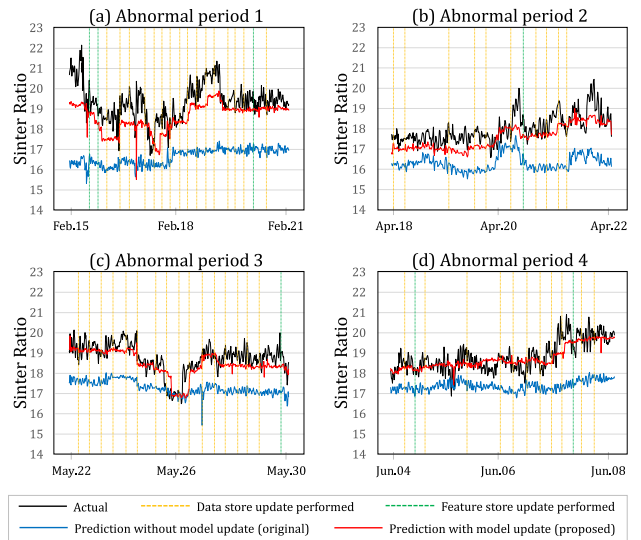


FIGURE 7. Model updates implemented during abnormal periods.

to the original state. Like the sudden drift, the proposed method actively performed feature store updates around the sharp drop near February 24. During the subsequent recovery to the original pattern, two additional feature store updates were conducted to adapt to ongoing fluctuations, demonstrating strong adaptability even in this recurring drift segment. This suggests that in regions with abrupt data shifts, feature reconfiguration is more effective, while in more stable regions, adjusting the retraining data window can serve as a suitable response strategy.

Interestingly, methods utilizing multiple windows, such as PL and AL, also contributed to the model's adaptability. Unlike single-window methods, multi-window structures retain stability through the long window while remaining sensitive to temporary changes via the short window. This makes them particularly effective in scenarios such as recurring drift, where temporary changes revert to the original pattern. Consequently, they can ensure both prediction stability and adaptability. In terms of performance, the MAPE values for PL were lower than that of the proposed method, while CTQ values for AL and PL remained stable within the 94–95% range, meeting the quality criteria. By simultaneously monitoring both long-term and short-term patterns and employing data store updates accordingly, these methods ensured a balanced response, maintaining both adaptability and stability under recurring drift conditions.

Abnormal period 3, located in the lower-left corner of Figure 7, corresponds to a gradual drift, where initial indications of pattern shift intensify before transitioning into a new structure. This type of drift is not only difficult to detect in its early stages but also involves an unstable and unpredictable transition phase, making it particularly challenging to manage. Around April 19, the proposed method responded to early signals by performing both feature store and data store updates. As the upward change began near April 20,

an additional data store update was initiated to accommodate the shift.

However, the evolving pattern was not adequately captured by existing predictive variables. In response to rising prediction errors, a sequence of feature store updates was triggered, allowing the model to select more appropriate features to reflect the new trend. The subsequent fluctuations and steadily increasing target values were effectively managed through ongoing data store updates. Finally, with the onset of a sharp decline, another feature store update was conducted to realign the model with the changing dynamics. As such, gradual drift requires more sensitive and rapid responses than other types of CD, which increases the risk of unnecessary retraining. Therefore, a more refined adaptation strategy—one that actively considers both data and feature reconfiguration—is essential to avoid overreaction and ensure model stability.

Interestingly, in this segment, AL, STEPDP, and DDM were found to be highly effective retraining strategies. Although they differ in mechanisms—AL uses multiple learners, STEPDP applies a sliding window, and DDM monitors error distributions—they share a common trait: retraining is triggered only after sufficient error accumulation. This suggests they are particularly well-suited for handling gradual drift, as they enable timely adaptation based on meaningful data shifts.

In contrast, methods such as PL and FHDDM showed lower performance. A possible reason is that these approaches detect CD based on error differences—between learners (PL) or current and minimum error rates (FHDDM). In gradual drift, where such differences increase slowly, these methods may suffer from delayed detection due to late threshold activation.

Abnormal period 4, in the lower-right corner of Figure 7, can be characterized as a sequence of sudden and incremental drift. As in the other segments, the proposed method initially performed data store updates in response to early signs of change, followed by feature store updates as the new pattern became more prominent. This pattern is particularly evident during the sharp drop immediately after May 7th, where the method responded with prompt updates, and in the subsequent stable phase. Following May 9th, an incremental upward trend emerged, which soon transitioned into a downward phase. In both cases, data store updates were carried out first, enabling the system to adjust to the evolving trend, and were subsequently complemented by feature store updates, demonstrating responsiveness to the continuously shifting data dynamics.

A notable finding here is that AL, STEPDP, and DDM did not show the same level of performance improvement under incremental drift as they did with gradual drift. This suggests that approaches relying on multiple window-based learners and fixed thresholds, or statistical assumptions may overlook subtle, continuous changes. In contrast, the proposed method not only compares long- and short-window models but also adds sensitivity by dynamically recombining input features.

This allows for a more consistent and responsive retraining strategy across both gradual and incremental drift scenarios.

Overall, the proposed model update strategy demonstrated strong predictive performance under frequent and irregular CDs. Through dynamic reconfiguration of predictor variables and active retraining, it effectively adapted to both temporary and gradual fluctuations, balancing robustness and adaptability. As a result, it achieved the highest overall performance within this case study, restoring a previously unreliable deployed model to a level of practical usability. Notably, this shows that even without replacing the model itself, long-term viability can be maintained by adjusting the retraining window and/or reconfiguring the predictor variables.

5) ABLATION STUDY

To more precisely evaluate the effectiveness of the proposed model management strategy, an ablation study was conducted. The proposed method consists of two key components: a feature store update, which dynamically reconfigures feature combinations, and a data store update, which constructs retraining datasets using recent data and assigns appropriate weights. The performance impact of including or excluding each component was analyzed, and the best predictive performance was achieved when both components were applied (Table 8). When the feature store update was omitted, the MAPE increased to 2.30 and the CTQ dropped to 92.71, indicating a decline in performance. Excluding the data store update resulted in a more pronounced performance degradation, with MAPE rising to 2.63 and CTQ decreasing to 88.77. These findings underscore the complementary roles of both update mechanisms in enhancing model accuracy and stability.

The relatively smaller performance degradation observed when the feature store update was omitted can be attributed to the structural characteristics of the predictive model used in this study—Lasso Regression. Lasso incorporates a regularization technique that drives the coefficients of less relevant features to zero, effectively performing automatic feature selection. This enables the model to adapt to a certain degree of data variation by emphasizing important features while suppressing less informative ones. In effect, feature reconfiguration within the existing feature set was implicitly performed during the data store update process, thereby fulfilling the role of a feature store update. Therefore, the observed performance gap likely reflects the added benefit of incorporating entirely new predictor variables into the feature store, which further enhanced model performance.

Unsurprisingly, more pronounced performance degradation was observed in segments where feature store updates were explicitly and frequently performed, suggesting the limitations of relying solely on Lasso's inherent feature selection mechanism during data store updates. For instance, in Abnormal period 1 and 4—characterized by sudden and incremental shifts—relatively larger declines in performance occurred despite active data store updates. This indicates that reweighting features within the existing set is not sufficient

TABLE 7. Results of ablation study.

Treatment	Predictive performance (MAPE/CTQ)				
	Overall (Jan.1. ~ Jun.11.)	Abnormal period 1 (Jan.31. ~ Feb.4.)	Abnormal period 2 (Feb.21. ~ Feb.27.)	Abnormal period 3 (Apr.18. ~ Apr.22.)	Abnormal period 4 (May.7. ~ May.10.)
With both feature and data store updates	2.14/93.12	2.41/90.42	1.72/94.08	1.94/94.92	2.06/91.19
Without feature store updates	2.30/93.71	2.79/88.79	1.82/96.06	1.66/97.00	2.48/88.26
Without data store updates	2.63/88.77	4.31/68.46	1.73/94.75	2.64/87.76	3.02/83.30

to maintain model accuracy when the explanatory power of those features has shifted. In such cases, although data store updates may offer partial adaptation, they fell short in addressing underlying structural changes. These findings highlight the inadequacy of data store updates alone in such contexts. A more precise and stable model update can be achieved when feature store updates are performed in conjunction, enabling the model to adapt not only to new data but also to evolving feature relevance.

The data store update was identified as a critical factor in improving predictive performance. When the data store update was omitted, the MAPE increased to 2.63 and the CTQ dropped to 88.77 over the full evaluation period. The performance degradation was particularly severe during Abnormal period 1, which involved a sudden drift: the MAPE rose to 4.31 and the CTQ fell to 68.46. Specifically, the most pronounced impact was observed during the sharp transition between February 2 and 3, where the feature store update was performed, but the absence of a data store update prevented the model from incorporating recent changes in the data. As a result, retraining was conducted based on outdated data, leading to a significant decline in predictive accuracy and an inability to respond effectively to the abrupt distributional shift. These findings underscore the crucial role of incorporating both post-drift data and recent segments where drift likely occurred in the retraining process, emphasizing that timely adaptation to emerging patterns is essential for maintaining model performance in dynamic environments.

The ablation results provide clear evidence that both components of the proposed framework are necessary to sustain performance under concept drift. When the feature store update is disabled, retraining is forced to rely on a fixed input representation, which limits the model's ability to respond to shifts in feature relevance and results in degraded accuracy when the explanatory structure of the process changes. Conversely, when the data store update is removed, the model is retrained with inadequately configured data that fails to emphasize the post-drift regime, making the update vulnerable to outdated history or transient noise. Only the full SHIFT design—jointly updating the feature store and the data store—consistently delivers the strongest and most

stable performance, indicating that the two mechanisms are complementary rather than interchangeable. While a single component may appear sufficient under mild or gradual changes, the ablation results show that this sufficiency breaks down under more challenging drift conditions (e.g., sudden or incremental shifts), where comprehensive adaptation requires simultaneous input reconfiguration and drift-aware retraining data construction.

6) DISCUSSION

Although the case study uses Lasso regression for field interpretability, SHIFT is base-model agnostic: it is an operational intelligence layer that specifies how a deployed model should be monitored, when it should be retrained, and how the feature store and data store should be revised— independent of whether the underlying predictor is a linear model or a Transformer. Importantly, SHIFT does not claim to make deep models inherently interpretable in the conventional sense; rather, it makes the model maintenance and self-evolution process interpretable by design. Each update is expressed through explicit, auditable actions—changes in retraining windows, buffer/weighting choices, and feature store reconfiguration—so practitioners can understand what changed, when, and why at the lifecycle level, even if the internal representation of the deep model remains complex. In this way, SHIFT generalizes naturally to deep models while preserving operational transparency and governance over continuous model evolution.

Beyond steel sintering VM, SHIFT can benefit other applications where feature relevance changes over time and retraining data must be curated rather than merely accumulated—e.g., remaining useful life estimation for rotating machinery, battery state of health prediction under seasonal and usage shifts, semiconductor process quality prediction, energy demand forecasting under regime changes, and anomaly prediction in wind turbines. For these applications, the main adjustable factors are: 1) how input variables are (re)constructed in the feature store (e.g., domain-specific feature groups, interaction candidates, sensor subsets, and constraints on introducing new variables), and 2) how dynamic retraining intervals are defined in the data store

(e.g., window lengths, buffer duration, weighting function, and thresholds reflecting drift severity and operational risk). To improve responsiveness in resource-constrained deployments, SHIFT can be simplified by using lightweight feature relevance estimators (e.g., incremental filter methods or low-cost importance proxies), restricting candidate feature recombination via domain rules, and adopting budget-aware retraining (trigger throttling, early stopping, or warm-start updates). Conversely, for higher-end deployments, the framework can be advanced by adding adaptive window optimization, context-aware drift signals (distributional and process-context indicators), and multi-objective orchestration that explicitly trades off predictive gain, retraining latency, and operational cost—thereby enabling scalable self-maintenance across diverse industrial AI settings.

Recent advances in industrial AI increasingly argue that the primary bottleneck is no longer modeling per se, but the absence of a systematic “foundation” that connects knowledge, data, and models into an operationalizable pipeline. This was explicitly framed as a unified industrial AI foundation framework in [66], in which the data module (preprocessing, data quality, feature engineering, visualization) and the model module (adaptation/generalization, deployment readiness) are coupled through iterative feedback rather than treated as a one-off, linear modeling step. From this perspective, CD in industrial settings is fundamentally a pipeline governance problem: reliable performance requires repeatable mechanisms to 1) update data/feature representations, 2) ensure the “AI-ready” status of retraining datasets, and 3) document and justify changes in an auditable manner—motivating orchestration layers that can translate drift evidence into concrete lifecycle actions rather than ad hoc retraining.

In parallel, the broader MLOps frontier has been converging on the view that competitive advantage increasingly depends on continuous delivery and automation pipelines and “MLOps-as-pipeline-automation,” where monitoring, retraining, and deployment are treated as standardized, repeatable operations rather than artisanal workflows. This trend becomes even more salient as foundation-model paradigms reduce the marginal effort of task-specific model building (via reuse and fine-tuning), shifting the center of gravity toward post-deployment operations: dataset/feature governance, drift-aware data selection, reproducibility, and transparent update logging. Accordingly, frameworks that automate data and model pipelines while remaining inspectable—i.e., making update triggers, data-window construction, feature reconfiguration, and rollout decisions explicit—are becoming essential for industrial AI, where stakeholders must balance responsiveness, stability, and operational cost under nonstationary regimes.

VI. CONCLUSION

This study highlights the importance of managing AI models as virtual engineering assets in complex and dynamic operational environments. Rather than treating predictive models

as static tools, we emphasize their role as evolving components within broader maintenance management systems. In this regard, the proposed approach—featuring dynamic updates to both the feature store and the data store—offers a structured yet flexible mechanism for sustaining model performance over time. Unlike prior studies that typically focus on retraining with fixed inputs or address feature/data handling in isolation, this work advances a robust retraining strategy by jointly orchestrating feature store updates and data store updates within a single unified framework. By adaptively refining both the inputs and the training datasets used in model retraining, the framework supports robust predictive accuracy even in the presence of CD. This lays the groundwork for intelligent, self-correcting decision support systems that enhance the reliability and resilience of engineering asset maintenance practices.

It is important to point out that the proposed method is a unified retraining framework that enables effective model maintenance without requiring model replacement in response to evolving data patterns. The proposed feature store update mechanism allows for flexible reconfiguration of input variable combinations during deployment, overcoming the limitations of conventional model maintenance approaches that rely on fixed predictor sets. By continuously reassessing feature relevance, the model can incorporate newly informative variables in a timely manner and adapt to CD more responsively. Complementing this, the data store update strategy ensures that retraining is grounded in meaningful post-drift data through the use of a buffer period and differential weighting, which together help distinguish genuine process shifts from transient noise. These strategies not only enhance predictive performance in the short term but also support the long-term reliability of AI models across various types of CD. Crucially, the proposed approach can be readily integrated into existing model deployment pipelines, functioning as an adaptive scheduler that enables continuous integration and continuous deployment (CI/CD) for model retraining—thereby offering a scalable and maintainable solution for industrial AI systems.

To validate its practical applicability, the proposed method was applied in a real-world steel manufacturing environment to maintain a VM model used for predicting sinter quality. The model, which had previously lost trust on the shop floor due to declining performance and biased predictions after deployment, was successfully stabilized without replacement. By dynamically reconfiguring the input feature set and adaptively adjusting the retraining window, the model not only recovered but also exceeded its original performance at the time of development. Compared to existing CD detection and adaptation approaches, the proposed method demonstrated superior accuracy and ranking across evaluation metrics, while maintaining low computational overhead. This efficiency facilitated seamless integration into the production environment, confirming its suitability for field-level deployment.

Despite the promising results, this study has several limitations that offer directions for future research. First, although the proposed method utilizes both long-term and short-term memory for CD detection, the window sizes were fixed throughout the experiments. This static configuration may have limited the method's ability to detect certain types of drift, particularly when the temporal dynamics of the data varied significantly. Future work should explore adaptive mechanisms to optimize window lengths in response to evolving drift patterns. Second, CD was identified solely based on changes in model performance, which may overlook latent signals not yet reflected in predictive accuracy. Integrating distribution-based or context-aware drift detection techniques could enhance the sensitivity and robustness of the framework. Third, this study did not consider the training and/or inference latency that inevitably arises in real-world deployments. Incorporating latency effects could reveal how detection and adaptation delays influence the overall model performance. Future studies could include sensitivity analyses to quantify these impacts and develop strategies for latency-aware retraining and adaptation. Finally, we acknowledge that in highly dynamic operational environments—where drift types, speeds, and regimes evolve rapidly—no single CD detection strategy is universally optimal, and performance-triggered mechanisms (including ours) may exhibit delayed or partial sensitivity. As a future work direction, we will strengthen SHIFT into a more drift-aware and deployment-ready orchestration layer by combining context-based drift signals and severity- and cost-aware update policies that automatically tune buffer duration, weighting strength, and feature reconfiguration intensity to balance responsiveness, stability, and computational latency in practice.

REFERENCES

- [1] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023.
- [2] G. Widmer and M. Kubat, "Learning in the presence of concept drift and hidden contexts," *Mach. Learn.*, vol. 23, no. 1, pp. 69–101, Apr. 1996.
- [3] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.
- [4] G. I. Webb, R. Hyde, H. Cao, H. L. Nguyen, and F. Petitjean, "Characterizing concept drift," *Data Mining Knowl. Discovery*, vol. 30, no. 4, pp. 964–994, Jul. 2016.
- [5] J. F. MacGregor and T. Kourti, "Statistical process control of multivariate processes," *Control Eng. Pract.*, vol. 3, no. 3, pp. 403–414, Mar. 1995.
- [6] T. C. Chang and F. F. Gan, "A cumulative sum control chart for monitoring process variance," *J. Quality Technol.*, vol. 27, no. 2, pp. 109–119, Apr. 1995.
- [7] J. M. Lucas and M. S. Saccucci, "Exponentially weighted moving average control schemes: Properties and enhancements," *Technometrics*, vol. 32, no. 1, pp. 1–12, Feb. 1990.
- [8] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Stat.*, vol. 22, no. 1, pp. 79–86, 1951.
- [9] F. J. Massey, "The Kolmogorov–Smirnov test for goodness of fit," *J. Amer. Stat. Assoc.*, vol. 46, no. 253, pp. 68–78, Mar. 1951.
- [10] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, nos. 1–2, pp. 100–115, Jun. 1954.
- [11] T. Dasu, S. Krishnan, S. Venkatasubramanian, and K. Yi, "An information-theoretic approach to detecting changes in multi-dimensional data streams," in *Proc. Symp. Interface Statist., Comput. Sci., Appl. (Interface)*, vol. 12, no. 2, 2006, pp. 1–22.
- [12] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Advances in Artificial Intelligence–SBIA* (Lecture Notes in Computer Science), vol. 3171, A. L. C. Bazzan and S. Labidi, Eds., Berlin, Germany: Springer, 2004, pp. 286–295.
- [13] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavalda, and R. Morales-Bueno, "Early drift detection method," in *Proc. 4th Int. Workshop Knowl. Discovery From Data Streams*, 2006, pp. 77–86.
- [14] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining*, Apr. 2007, pp. 443–448.
- [15] P. Porwik, K. Wrobel, T. Orczyk, and R. Doroz, "FBDD: Feature-based drift detector for batch processing data," *Cluster Comput.*, vol. 27, no. 5, pp. 6805–6822, Aug. 2024.
- [16] F. Hinder, V. Vaquet, and B. Hammer, "Feature-based analyses of concept drift," *Neurocomputing*, vol. 600, Oct. 2024, Art. no. 127968.
- [17] H. Zhang, X. Chen, M. Hu, and V. Sugumaran, "Multilayer concept drift detection method based on model explainability," *IEEE Access*, vol. 12, pp. 190791–190808, 2024.
- [18] B. Zimmermann and M. Boussard, "Improving drift detection by monitoring Shapley loss values," in *Proc. Int. Conf. Pattern Recognit. Artif. Intell. Cham, Switzerland*: Springer, 2022, pp. 455–466.
- [19] L. Yang, J. Cheng, Y. Luo, T. Zhou, and X. Zhang, "Detecting and rationalizing concept drift: A feature-level approach for understanding cause–effect relationships in dynamic environments," *Expert Syst. Appl.*, vol. 260, Jan. 2024, Art. no. 125365.
- [20] X. Zheng, B. Aragam, P. Ravikumar, and E. P. Xing, "DAGs with NO TEARS: Continuous optimization for structure learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 9472–9483.
- [21] R. Elwell and R. Polikar, "Incremental learning of concept drift in non-stationary environments," *IEEE Trans. Neural Netw.*, vol. 22, no. 10, pp. 1517–1531, Oct. 2011.
- [22] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, Apr. 2014.
- [23] F. Bayram, B. S. Ahmed, and A. Kassler, "From concept drift to model degradation: An overview on performance-aware drift detectors," *Knowl.-Based Syst.*, vol. 245, Jun. 2022, Art. no. 108632.
- [24] C. Renggli, L. Rimanic, N. M. Gürel, B. Karlaš, W. Wu, and C. Zhang, "A data quality-driven view of MLOps," 2021, *arXiv:2102.07750*.
- [25] S. Mei, C. Liu, Q. Wang, and H. Su, "Model provenance management in MLOps pipeline," in *Proc. 8th Int. Conf. Comput. Data Eng.*, Jan. 2022, pp. 45–50.
- [26] S. G. Soares and R. Araújo, "An on-line weighted ensemble of regressor models to handle concept drifts," *Eng. Appl. Artif. Intell.*, vol. 37, pp. 392–406, Jan. 2015.
- [27] S. G. Soares and R. Araújo, "A dynamic and on-line ensemble regression for changing environments," *Expert Syst. Appl.*, vol. 42, no. 6, pp. 2935–2948, Apr. 2015.
- [28] D. Brzezinski and J. Stefanowski, "Combining block-based and online methods in learning ensembles from concept drifting data streams," *Inf. Sci.*, vol. 265, pp. 50–67, May 2014.
- [29] L. Du, Q. Song, and X. Jia, "Detecting concept drift: An information entropy based method using an adaptive sliding window," *Intell. Data Anal.*, vol. 18, no. 3, pp. 337–364, Apr. 2014.
- [30] H. Guo, H. Li, Q. Ren, and W. Wang, "Concept drift type identification based on multi-sliding windows," *Inf. Sci.*, vol. 585, pp. 1–23, Mar. 2022.
- [31] T. Schröder and M. Schulz, "Monitoring machine learning models: A categorization of challenges and methods," *Data Sci. Manage.*, vol. 5, no. 3, pp. 105–116, Sep. 2022.
- [32] P. Liang, B. Song, X. Zhan, Z. Chen, and J. Yuan, "Automating the training and deployment of models in MLOps by integrating systems with machine learning," 2024, *arXiv:2405.09819*.
- [33] K. Nishida and K. Yamauchi, "Detecting concept drift using statistical testing," in *Proc. Int. Conf. Discovery Sci.*, 2007, pp. 264–269.
- [34] I. Khamassi and M. Sayed-Mouchaweh, "Drift detection and monitoring in non-stationary environments," in *Proc. IEEE Conf. Evolving Adapt. Intell. Syst. (EAIS)*, Jun. 2014, pp. 1–6.

- [35] S. Sakthithasan, R. Pears, and Y. S. Koh, "One pass concept change detection for data streams," in *Proc. 17th Pacific-Asia Conf. Adv. Knowl. Discovery Data Mining*, Apr. 2013, pp. 461–472.
- [36] D. T. J. Huang, Y. S. Koh, G. Dobbie, and R. Pears, "Detecting volatility shift in data streams," in *Proc. IEEE Int. Conf. Data Mining*, Dec. 2014, pp. 863–868.
- [37] D.-J. Lim, S. Kim, U. Hwang, S. Jeong, D. Kim, and Y. Kim, "Development of a virtual metrology system for smart manufacturing: A case study of spandex fiber production," *Comput. Ind.*, vol. 145, Feb. 2023, Art. no. 103825.
- [38] S. H. Bach and M. A. Maloof, "Paired learners for concept drift," in *Proc. 8th IEEE Int. Conf. Data Mining*, Dec. 2008, pp. 23–32.
- [39] Y. Xu, R. Xu, W. Yan, and P. Ardis, "Concept drift learning with alternating learners," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, May 2017, pp. 2104–2111.
- [40] A. Liu, G. Zhang, K. Wang, and J. Lu, "Fast switch Naïve Bayes to avoid redundant update for concept drift learning," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2020, pp. 1–7.
- [41] A. Azeem, I. Ismail, S. S. Mohani, K. U. Danyaro, U. Hussain, S. Shabbir, and R. Z. Bin Jusoh, "Mitigating concept drift challenges in evolving smart grids: An adaptive ensemble LSTM for enhanced load forecasting," *Energy Rep.*, vol. 13, pp. 1369–1383, Jun. 2025.
- [42] Z. Chen, M. Han, H. Wu, M. Li, and X. Zhang, "A multi-level weighted concept drift detection method," *J. Supercomput.*, vol. 79, no. 5, pp. 5154–5180, Mar. 2023.
- [43] V. Cerqueira, H. M. Gomes, A. Bifet, and L. Torgo, "STUDD: A student–teacher method for unsupervised concept drift detection," *Mach. Learn.*, vol. 112, no. 11, pp. 4351–4378, Nov. 2023.
- [44] S. G. Soares and R. Araújo, "An adaptive ensemble of on-line extreme learning machines with variable forgetting factor for dynamic system prediction," *Neurocomputing*, vol. 171, pp. 693–707, Jan. 2016.
- [45] D. R. D. L. Cabral and R. S. M. D. Barros, "Concept drift detection based on Fisher's exact test," *Inf. Sci.*, vols. 442–443, pp. 220–234, May 2018.
- [46] R. S. M. Barros, D. R. L. Cabral, P. M. Gonçalves, and S. G. T. C. Santos, "RDDM: Reactive drift detection method," *Expert Syst. Appl.*, vol. 90, pp. 344–355, Dec. 2017.
- [47] R. A. Coelho, L. C. B. Torres, and C. L. de Castro, "Concept drift detection with quadtree-based spatial mapping of streaming data," *Inf. Sci.*, vol. 625, pp. 578–592, May 2023.
- [48] A. Pesaraghader, H. L. Viktor, and E. Paquet, "McDiarmid drift detection methods for evolving data streams," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2018, pp. 1–9.
- [49] A. Liu, G. Zhang, and J. Lu, "Fuzzy time windowing for gradual concept drift adaptation," in *Proc. IEEE Int. Conf. Fuzzy Syst. (FUZZ-IEEE)*, Jul. 2017, pp. 1–6.
- [50] I. Baidari and N. Honnikoll, "Bhattacharyya distance based concept drift detection method for evolving data stream," *Expert Syst. Appl.*, vol. 183, Nov. 2021, Art. no. 115303.
- [51] A. Pesaraghader and H. L. Viktor, "Fast Hoeffding drift detection method for evolving data streams," in *Proc. Mach. Learn. Knowl. Discovery Databases, Eur. Conf.*, Riva del Garda, Italy, Sep. 2016, pp. 96–111.
- [52] I. Frías-Blanco, J. del Campo-Ávila, G. Ramos-Jiménez, R. Morales-Bueno, A. Ortiz-Díaz, and Y. Caballero-Mota, "Online and non-parametric drift detection methods based on Hoeffding's bounds," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 3, pp. 810–823, Mar. 2015.
- [53] G. J. Ross, N. M. Adams, D. K. Tasoulis, and D. J. Hand, "Exponentially weighted moving average charts for detecting concept drift," *Pattern Recognit. Lett.*, vol. 33, no. 2, pp. 191–198, Jan. 2012.
- [54] I. Khamassi, M. Sayed-Mouchaweh, M. Hammami, and K. Ghédira, "Self-adaptive windowing approach for handling complex concept drift," *Cognit. Comput.*, vol. 7, no. 6, pp. 772–790, Dec. 2015.
- [55] R. Pears, S. Sakthithasan, and Y. S. Koh, "Detecting concept change in dynamic data streams: A sequential approach based on reservoir sampling," *Mach. Learn.*, vol. 97, no. 3, pp. 259–293, Dec. 2014.
- [56] M. Harel, S. Mannor, R. El-Yaniv, and K. Crammer, "Concept drift detection through resampling," in *Proc. Int. Conf. Mach. Learn.*, 2014, pp. 1009–1017.
- [57] A. Kahraman, M. Kantardzic, and M. Kotan, "Dynamic modeling with integrated concept drift detection for predicting real-time energy consumption of industrial machines," *IEEE Access*, vol. 10, pp. 104622–104635, 2022.
- [58] P. R. Winters, "Forecasting sales by exponentially weighted moving averages," *Manage. Sci.*, vol. 6, no. 3, pp. 324–342, Apr. 1960.
- [59] E. S. Gardner, "Exponential smoothing: The state of the art," *J. Forecasting*, vol. 4, no. 1, pp. 1–28, Jan. 1985.
- [60] A. S. Rahman, M. Lee, J. Lim, Y. Cho, and C. Shin, "A prediction model for steel factory manufacturing product based on energy consumption using data mining technique," *J. Sci. Eng. Manag.*, vol. 1, no. 2, pp. 9–16, 2020.
- [61] V. E. Sathishkumar, J.-H. Lim, M. Lee, K. Cho, J. Park, and C.-S. Shin, "Industry energy consumption prediction using data mining techniques," *Int. J. Energy Inf. Commun.*, vol. 11, no. 1, pp. 7–14, 2020.
- [62] U. M. Fayyad and K. B. Irani, "Multi-interval discretization of continuous-valued attributes for classification learning," in *Proc. IJCAI*, vol. 2, 1993, pp. 1022–1027.
- [63] H. Peng and C. Ding, "Minimum redundancy and maximum relevance feature selection and recent advances in cancer classification," *Feature Sel. Data Min.*, vol. 52, pp. 57–112, Apr. 2005.
- [64] R. Kerber, "ChiMerge: Discretization of numeric attributes," in *Proc. 10th Nat. Conf. Artif. Intell.*, 1992, pp. 123–128.
- [65] Z. Zeng, H. Zhang, R. Zhang, and C. Yin, "A novel feature selection method considering feature interaction," *Pattern Recognit.*, vol. 48, no. 8, pp. 2656–2666, Aug. 2015.
- [66] J. Lee and H. Su, "Rethinking industrial artificial intelligence: A unified foundation framework," 2025, *arXiv:2504.01797*.



SEONG-HYUN SEO received the B.S. degree in data science and systems management engineering and the M.S. degree in industrial engineering from Sungkyunkwan University, South Korea. She is currently an AI/ML Researcher Intern with Boeing AI, BKETC, Seoul, South Korea. Her research interests include data science, machine learning, artificial intelligence, multimodal time series, physical AI, and manufacturing process optimization.



DONG-JOON LIM received the B.S. and M.S. degrees in industrial engineering from Sungkyunkwan University, South Korea, and the Ph.D. degree in engineering and in technology management from Portland State University, USA. He is currently an Associate Professor with the Department of Systems Management Engineering, Sungkyunkwan University, South Korea, and the Founder and the CEO of Technometrics Inc., a company specializing in MLOps optimization solutions. His research interests include technological forecasting, optimization modeling, productivity analysis, and data science. He is also a Developer of the open-source R package "DJL," which implements various decision-support tools related to econometrics and technometrics. His academic honors include the Emerald Literati Network Award; the Marie Brown Award; and various fellowships from PSU, SKKU, and A&P.

...